

JOINS en SQL

Cuando tenemos todos los datos en una sola tabla, acceder a la información que necesitamos es bastante simple: solo buscamos la fila adecuada y listo. Pero cuando una empresa empieza a crecer, sus datos también lo hacen, y las cosas se complican un poco. Lo más común es que esos datos terminen organizados en varias tablas dentro de una base de datos relacional. En esos casos, antes de poder trabajar con la información, tenemos que unir esas tablas.

Para eso existen las sentencias JOIN en SQL. Nos ayudan a conectar registros de dos tablas diferentes usando una clave que ambas comparten. Normalmente, esto se hace utilizando una combinación de claves primarias y foráneas que permiten establecer relaciones entre las tablas de forma lógica.

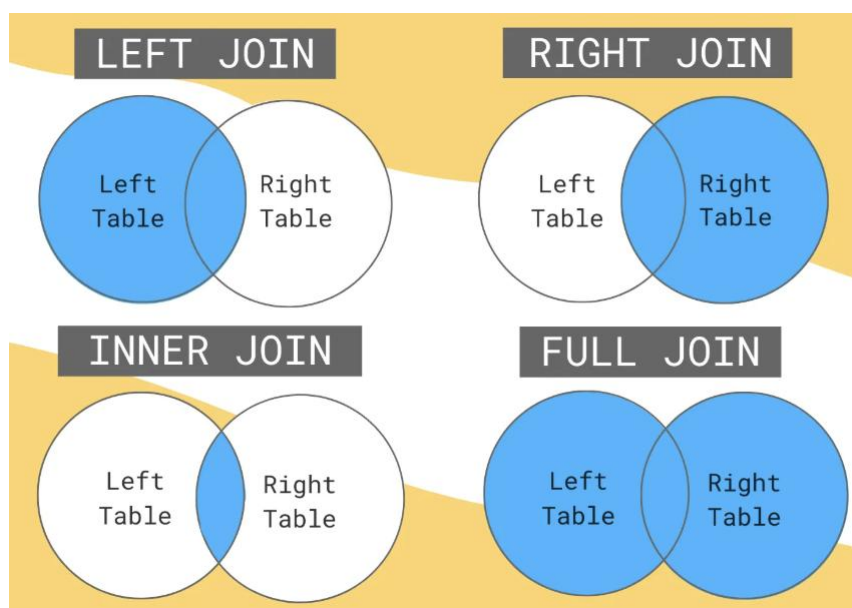


Ilustración 1 <https://learnsql.es/blog/como-aprender-a-hacer-joins-en-sql/>

1. INNER JOIN: Qué es y cómo funciona

Las sentencias INNER JOIN permiten combinar registros de dos tablas diferentes cuando existe una coincidencia entre ellas, normalmente a través de una relación de clave primaria y clave foránea.

Esto permite realizar consultas que integran datos de ambas tablas en un mismo resultado.

● **Muy importante:** Un INNER JOIN **solo devuelve las filas que tienen coincidencia en ambas tablas**. Si no hay un valor relacionado en una de ellas, **esa fila no aparecerá** en el resultado.

Sintaxis general

```
SELECT tabla1.columna1, tabla2.columna2, ...  
FROM tabla1  
INNER JOIN tabla2 ON tabla1.columna_comun = tabla2.columna_comun;
```

Notas importantes:

- La tabla escrita en la cláusula FROM es considerada la **tabla izquierda** o **base**.
- En el SELECT pueden aparecer columnas de **ambas tablas**, siempre que estén correctamente enlazadas mediante la condición del JOIN.
- Aunque se puede usar SELECT * para mostrar todas las columnas de ambas tablas, **se recomienda especificar solo las columnas necesarias**, para mayor claridad y eficiencia.

Ejemplo:

```
SELECT pedidos.idpedido, pedidos.total, clientes.nombre  
FROM pedidos  
INNER JOIN clientes ON pedidos.idcliente = clientes.idcliente;
```

2. ¿JOIN o INNER JOIN?

En SQL, JOIN y INNER JOIN son equivalentes. Si escribes simplemente JOIN, el sistema entiende por defecto que es un INNER JOIN.

Ejemplo:

```
SELECT pedidos.idpedido, clientes.nombre  
FROM pedidos  
JOIN clientes ON pedidos.idcliente = clientes.idcliente;
```

No obstante, se recomienda usar siempre INNER JOIN para mantener la claridad, especialmente cuando se empiecen a utilizar otros tipos de JOIN como LEFT JOIN o RIGHT JOIN.

3. INNER JOIN con múltiples tablas

Puedes enlazar más de dos tablas utilizando varios INNER JOIN, siempre que haya una relación entre ellas:

Sintaxis general

```
SELECT tabla1.columna1, tabla2.columna2, tabla3.columna3  
FROM tabla1  
INNER JOIN tabla2 ON tabla1.id = tabla2.id_tabla1  
INNER JOIN tabla3 ON tabla2.id = tabla3.id_tabla2;
```

Este tipo de consultas es común cuando los datos están distribuidos en distintas tablas por diseño normalizado.

Ejemplo práctico con la tienda Alislow

Supongamos que añadimos una tercera tabla llamada productos, donde se indica qué producto se ha comprado en cada pedido. La tabla pedidos incluirá ahora una columna idproducto.

Queremos obtener una lista con el nombre del cliente, el producto comprado y la fecha del pedido. En este caso, la sintaxis sería:

Ejemplo:

```
SELECT clientes.nombre, productos.nombre, pedidos.fecha  
FROM pedidos  
INNER JOIN clientes ON pedidos.idcliente = clientes.idcliente  
INNER JOIN productos ON pedidos.idproducto = productos.idproducto;
```

Este INNER JOIN múltiple une:

- pedidos con clientes (por idcliente)
- pedidos con productos (por idproducto)

El resultado solo incluirá los pedidos que tengan un cliente y un producto válidos en ambas tablas

4. Unir tablas usando WHERE

Antes de la sintaxis moderna de JOIN, era habitual unir tablas mediante la cláusula WHERE:

Ejemplo:

```
SELECT pedidos.idpedido, clientes.nombre  
FROM pedidos, clientes  
WHERE pedidos.idcliente = clientes.idcliente;
```

Aunque funcional, este enfoque **mezcla condiciones de unión y de filtrado**, lo que puede llevar a confusión o errores en consultas más complejas. Por ello, **se desaconseja su uso** en favor de la sintaxis JOIN.

5. Filtrar resultados tras hacer un JOIN

Una vez que se han unido dos o más tablas mediante INNER JOIN, se puede aplicar una cláusula WHERE para mostrar sólo las filas que cumplen ciertas condiciones.

Ejemplo:

```
SELECT pedidos.idpedido, pedidos.total, clientes.nombre  
FROM pedidos  
INNER JOIN clientes ON pedidos.idcliente = clientes.idcliente  
WHERE pedidos.total > 100;
```

En este ejemplo se muestran solo los pedidos mayores de 100 euros, manteniendo separados el ON (para enlazar tablas) y el WHERE (para filtrar resultados). Esta separación es una buena práctica en SQL.